# Benchmarking Large Language Models on Repairing Qiskit Programs using Bugs4Q

**1. Motivación y Problema Principal**

- **Inestabilidad de los benchmarks cuánticos:** Los autores descubren que benchmarks como Bugs4Q sufren de una *inversión silenciosa de etiquetas*. A medida que el framework (Qiskit) se actualiza, los programas con fallos dejan de reproducir el error o las correcciones de referencia dejan de ejecutarse sin dar mensajes de error evidentes.
- Por ello, la validez del benchmark depende de la combinación exacta del par (caso de prueba, versión de Qiskit) y no del benchmark por sí solo.

**2. Metodología**

- **Construcción del benchmark:** Se combinaron dos conjuntos de datos de Bugs4Q (Bugs4Q-Framework y Bugs4Q-NA), consolidando **67 defectos reales de Qiskit** deduplicados.
- **Creación de tests ejecutables:** Se escribieron y revisaron oráculos de prueba ejecutables basados en el módulo bajo prueba (MUT) para evaluar si el código reparado funciona correctamente.
- **Evaluación fijada por versión:** Validaron cada caso bajo **6 versiones específicas de Qiskit**: 0.25.0, 0.45.0, 1.0.0, 1.1.1, 2.0.0 y 2.3.1.
- **Modelos evaluados:** Se probaron cuatro LLMs:
    - GPT-4o-mini
    - GPT-5o-mini
    - GPT-5.4
    - GPT-5.4-mini (Generando hasta 10 candidatos de reparación por cada defecto).

**3. Resultados Clave**

1. **Rendimiento de los modelos (Métrica pass@10 sin filtrar por validez):**
    - **GPT-5.4:** Obtuvo la tasa de éxito más alta con un **48.8%**.
    - **GPT-5.4-mini: 47.3%**.

- **GPT-5o-mini: 30.3%**.
- **GPT-4o-mini: 22.6%**.

2. **Impacto de la versión de Qiskit:**
    - Todos los modelos alcanzaron su punto máximo de rendimiento en la versión **Qiskit 0.45.0** y sufrieron una **caída drástica a partir de la transición a Qiskit 1.0** y versiones posteriores (2.x).
3. **Incompatibilidad del entorno vs. Lógica de reparación:**
    - Un gran porcentaje de las fallas al reparar código no se debe a lógica errónea, sino a **APIs obsoletas o incompatibles** que cambiaron entre versiones. La incompatibilidad representó entre el **13% y el 56%** de los fallos según el modelo.
    - El **64%** de las reparaciones exitosas registradas ocurrieron en entradas del benchmark que eran **invalidadas** en la versión de prueba.

**4. Conclusión y Recomendaciones**

- Evaluar modelos de reparación automática de programas (APR) cuánticos con métricas genéricas y agnósticas de la versión distorsiona los resultados.
- Se propone que las evaluaciones de APR cuántico adopten de manera estándar protocolos con **versiones fijas del entorno** y un **seguimiento explícito de la ejecutabilidad**.
- Los autores liberaron el conjunto de datos revalidado y fijado por versión en su repositorio de GitHub.

# On Repairing Quantum Programs Using ChatGPT

**1. Contexto y Motivación**

- **El desafío cuántico:** La programación cuántica presenta desafíos únicos debido a principios como la superposición, el entrelazamiento y la presencia de ruido en dispositivos de escala intermedia (NISQ). Esto genera tipos de errores específicos (en puertas cuánticas, mediciones, etc.) que son difíciles de detectar y corregir manualmente.
- **La brecha en APR:** Mientras que la reparación automática de código clásico con modelos de lenguaje (LLMs) ha avanzado significativamente, en la computación cuántica aún faltan técnicas automatizadas dedicadas.

**2. Metodología y Diseño Experimental**

- **Benchmark:** Se utilizaron **38 errores reales y reproducibles** del benchmark **Bugs4Q**, programados en el framework **Qiskit** de IBM.
- **Evaluación de prompts (4 enfoques):**
    1. Chat: Se proporciona únicamente el código con fallos (sin descripción).
    2. Chat_D: Código + una descripción básica del problema.
    3. Chat_2D: Código + una descripción detallada (intención del programa y comportamiento esperado).
    4. Chat_3D: Mismo contenido que Chat_2D, pero informando directamente a ChatGPT que el programa contiene errores sin preguntarle si existe un bug.
- **Proceso de diálogo:** Si el primer parche fallaba al ejecutarse o no pasaba las pruebas, se le proporcionaba a ChatGPT el mensaje de error del compilador o la información del fallo del test en diálogos iterativos.
- **Métricas:** Se evaluó el desempeño usando TOP-5 (si al menos uno de los 5 intentos corrige el error) y AVG-5 (tasa de éxito promedio entre las 5 iteraciones).

**3. Resultados Principales**

- **Eficacia general:** ChatGPT logró reparar con éxito **29 de los 38 errores** (un **76.3%** de tasa de éxito general en TOP-5) cuando se le brindaron descripciones detalladas y retroalimentación de las pruebas.

- **Impacto del nivel de detalle en el prompt:**
    - **Solo código (Chat):** Solo reparó 10 de 38 errores.
    - **Con información detallada (Chat_2D y Chat_3D):** Lograron la tasa más alta reparando 29 de 38 errores.
- **Desempeño por tipo de error (con Chat_3D):**
    - **Errores clásicos:** 100% de éxito en TOP-5 (6/6) y 97% en AVG-5.
    - **Errores cuánticos simples:** 84% de éxito en TOP-5 (21/25) y 67% en AVG-5.
    - **Errores de algoritmos cuánticos complejos:** Solo reparó el 28.5% en TOP-5 (2/7) y 17% en AVG-5. El modelo falló principalmente en algoritmos cuánticos propios/personalizados que no formaban parte de su corpus de entrenamiento público.

**4. Conclusiones y Discusión**

- **Factibilidad:** El estudio demuestra que es plenamente factible utilizar LLMs como ChatGPT para asistir en la reparación de código cuántico.
- **Herramienta semiautomática:** ChatGPT actúa como un sistema semiautomático. Su efectividad depende drásticamente de la capacidad del desarrollador para describir el error y proporcionar el contexto adecuado en el prompt.
- **Limitaciones:** El modelo muestra deficiencias en errores altamente complejos o en algoritmos cuánticos novedosos sobre los cuales no fue entrenado previamente.

# A UNIFIED MULTI-FRAMEWORK BENCHMARK FOR LLM-BASED QUANTUM CODE GENERATION

**1. Contexto y Problema**

- **Aislamiento de habilidades:** La mayoría de los benchmarks existentes evalúan la generación de código cuántico en un solo framework (p. ej., solo Qiskit). Esto dificulta distinguir si el éxito de un LLM se debe a un **razonamiento cuántico conceptual** o a la simple **familiaridad con las APIs/sintaxis** de un entorno específico.
- **Naturaleza probabilística:** A diferencia del código clásico (que se evalúa de forma determinista), el código cuántico suele producir distribuciones de probabilidad en las mediciones, requiriendo métricas de evaluación adecuadas.

**2. La Propuesta: QuanBench+**

- **Estructura unificada:** Adapta **42 tareas funcionales** para que se ejecuten y evalúen de manera idéntica en los tres principales frameworks cuánticos: **Qiskit**, **PennyLane** y **Cirq**.
- **Categorías de tareas:** Algoritmos cuánticos (31 tareas), Preparación de estados (6 tareas) y Descomposición de puertas (5 tareas).
- **Criterio de corrección:**
    - Evaluación funcional mediante ejecución directa de código.
    - Uso de la **divergencia KL (Kullback-Leibler)** con un umbral de $0.05$ para validar si la distribución de salida del código generado coincide con la solución canónica esperada.
    - Exclusión explícita de la métrica de *fidelidad de estado/unitaria* al generar falsos negativos en circuitos optimizados o transpilados que son funcionalmente equivalentes.

**3. Preguntas de Investigación y Resultados Clave**

**RQ1: Precisión en la generación directa (Pass@1)**

- **Dificultad asimétrica:** Existe una notable brecha de rendimiento entre frameworks. **Qiskit** resultó ser consistentemente el objetivo más fácil para los modelos, **Cirq** el intermedio, y **PennyLane** el más difícil.

- **Puntuaciones máximas en Pass@1 (generación directa a la primera):**
    - **Qiskit:** 59.5% (Gemini 3 Pro)
    - **Cirq:** 54.8% (Gemini 3 Pro)
    - **PennyLane:** 42.9% (GPT-5.1)
- **Conclusión:** Aunque los modelos frontier muestran avances, la generación fiable entre múltiples frameworks sigue sin resolverse y depende enormemente del framework evaluado.

**RQ2: Efecto del andamiaje/código base (Prefill)**

- Incluir importaciones y firmas de funciones en el prompt (prefill) reduce los errores de sintaxis y la fricción de interfaz, ayudando principalmente a modelos medianos o pequeños. Sin embargo, no resuelve los fallos profundos de lógica de programación.

**RQ3: Impacto de la reparación basada en retroalimentación (Pass@1 FB)**

- Se permitió a los modelos corregir su código en hasta 5 intentos cuando la ejecución fallaba (por errores de sintaxis o respuestas incorrectas en el test).
- **Resultados tras la retroalimentación (Pass@1 FB):**
    - **Qiskit:** Aumentó hasta **83.3%** (GPT-5.1).
    - **Cirq:** Aumentó hasta **76.2%** (Gemini 3 Pro).
    - **PennyLane:** Aumentó hasta **66.7%** (GPT-5.1 y Gemini 3 Pro).
- **Análisis de errores:** El bucle de retroalimentación elimina eficazmente los errores de sintaxis (que caen de un 4.7% a un 1.5%) e importaciones fallidas. No obstante, los fallos residuales que persisten tras la reparación quedan fuertemente dominados por **respuestas incorrectas (53.4%)** y **errores de lógica (22.0%)**.

**4. Conclusiones**

El estudio demuestra que las capacidades de los LLMs actuales para programar software cuántico están fuertemente condicionadas por la memoria o exposición que tienen a las APIs de un framework específico (mayor en Qiskit) más que por un razonamiento cuántico verdaderamente transferible. Aunque la retroalimentación automatizada durante el tiempo de ejecución mejora sustancialmente la tasa de

éxito, superar el rendimiento actual requerirá un mejor entrenamiento en razonamiento composicional cuántico.

# Explainable Quantum Program Repair with Verifiable Proof Traces

1. Problema y Motivación

- Limitación de los LLMs actuales: Aunque los LLMs generan parches y explicaciones en lenguaje natural para corregir código, estas explicaciones suelen ser informales, evaluadas tras la generación y no comprobables.
- Sensibilidad cuántica: En la computación cuántica, pequeñas variaciones sintácticas pueden alterar completamente la semántica del circuito (equivalencia de puertas, fidelidad del estado, etc.). Las explicaciones exclusivamente en texto no garantizan la corrección semántica.
- Solución propuesta: Tratar las explicaciones no como texto libre, sino como objetos ejecutables y verificables mediante trazas de prueba (*proof traces*).

2. Metodología del Marco de Trabajo

El sistema opera en tres etapas:

1. Generación de candidatos y justificaciones estructuradas: Dado un programa cuántico defectuoso (en QASM, Qiskit o Cirq) y una especificación, un LLM propone parches candidatos junto con justificaciones estructuradas (pasos de transformación como reordenamiento de puertas o sustituciones).
2. Compilación a trazas de prueba y verificación: Las justificaciones se compilan en trazas de prueba ejecutables. Luego se validan con múltiples *backends* de verificación cuántica formal:
    - Verificación de equivalencia de circuitos.
    - Razonamiento con ZX-calculus.
    - Análisis de estabilizadores (para circuitos de Clifford).
    - Simulación cuántica (para estimar fidelidad).
3. Selección de reparaciones fieles: Solo se aceptan y seleccionan las soluciones cuyas explicaciones son 100% verificadas por los motores formales.

3. Evaluación Experimental

- Benchmark: Se utilizaron 500 circuitos de QASMBench (de 2 a 130 qubits).

- Mutaciones: Se generaron 5,000 circuitos con errores mediante 5 tipos de mutaciones (eliminación/inserción de puertas, reordenamiento, perturbación de parámetros y alteración de índices de qubits).
- Resultados principales:
    - Tasa de éxito en reparación: 76.1%, superando a otros métodos basados en LLMs (como *LLM-QAPR* con 72.9% o *HornBro* con 74.6%).
    - Precisión semántica: Alcanzó un 96.1%, lo que significa que casi todas las soluciones aceptadas eran verdaderamente equivalentes al circuito original, evitando parches falsos o espurios.
    - Fidelidad de la explicación (*Faithfulness*): Logró un 94.8% de explicaciones verificadas, frente a tan solo el 18.6% que logran las explicaciones en lenguaje natural tradicionales.
    - Distancia de edición: Tuvo el promedio más bajo (0.28), generando cambios mínimos en el código original.

4. Conclusiones Clave

- Obligar a que las explicaciones sean verificables no reduce la tasa de éxito del modelo si se generan y filtran múltiples candidatos, y mejora drásticamente la precisión semántica del código reparado.
- ZX-calculus demostró ser el motor de verificación más crítico para razonar sobre la equivalencia y reescritura de circuitos.
- El enfoque demuestra que basar las explicaciones en semántica ejecutable y verificable es clave para construir herramientas de IA fiables para el desarrollo de software cuántico.

# Large Language Models for Quantum Software Engineering: Opportunities and Challenges in Modeling, Generation, Testing, and Repair

**1. Las 4 Áreas Clave de Aplicación (Oportunidades)**

El artículo estructura el uso de los LLMs en cuatro tareas fundamentales del desarrollo de software cuántico:

- **A. Modelado y Análisis de Programas Cuánticos:**
    - *Oportunidades:* Resumir la intencionalidad y semántica de circuitos cuánticos en lenguaje natural, resumir patrones de errores complejos, búsqueda semántica de código, generación de modelos formales (como grafos de dependencia cuántica) y traducción cruzada entre SDKs (Qiskit, Cirq, Q#).
- **B. Generación de Código Cuántico:**
    - *Oportunidades:* Traducir descripciones algorítmicas en lenguaje natural a código ejecutable, recomendar/diseñar estructuras de Circuitos Cuánticos Parametrizados (PQC) para algoritmos como VQE o QAOA y coordinar flujos de trabajo híbridos (cuántico-clásicos).
- **C. Pruebas (Testing) de Programas Cuánticos:**
    - *Oportunidades:* Generar casos de prueba inteligentes (creando circuitos cuánticos válidos para explorar casos límite), asistir en la construcción de oráculos de prueba y aserciones e identificar patrones de errores específicos del dominio cuántico.
- **D. Reparación Automática de Programas (APR):**
    - *Oportunidades:* Generación automática de parches analizando registros de ejecución, coordinación de flujos de trabajo de reparación de extremo a extremo y asistencia en reparaciones conscientes del hardware.

**2. Desafíos Únicos del Dominio Cuántico**

Aplicar LLMs en QSE no es una simple adaptación del software clásico; enfrenta obstáculos físicos y computacionales:

1. **Escasez de Datos de Entrenamiento de Alta Calidad:** Falta de grandes repositorios de código cuántico etiquetado o pares de error-corrección.
2. **Brecha de Corrección Semántica y Alucinaciones:** Código sintácticamente correcto en OpenQASM/Qiskit suele contener errores semánticos destructivos que alteran la evolución del estado cuántico.
3. **El Problema del Oráculo de Prueba:** Dado que el estado cuántico colapsa al medirse y los resultados son probabilísticos, verificar la corrección de un programa o parche requiere inferencia estadística sobre miles de ejecuciones (*shots*).
4. **La Brecha Simulación-Hardware (Sim-to-Real Gap):** Pruebas y reparaciones validadas en simuladores ideales (sin ruido) suelen fallar en procesadores cuánticos reales (NISQ) debido a decoherencia, errores de puertas y conectividad limitada de qubits.
5. **Alineación con Restricciones Físicas:** Optimizar un circuito exige reducir la profundidad y las puertas CNOT para luchar contra el ruido físico y adaptarse a la topología real del chip.

**3. Hoja de Ruta Gradual (Roadmap hacia 2030)**

Los autores proponen una agenda de investigación en tres fases para la comunidad:

- **Objetivos a Corto Plazo (2025–2026) – Problemas Fundamentales:**
    - Síntesis y aumentación de datos; construcción de benchmarks unificados.
    - Integración de LLMs con métodos formales de verificación e ingenierías de prompt tipo *Chain-of-Thought* orientadas a la física cuántica.
    - Abordar el problema del oráculo mediante pruebas metamórficas, basadas en propiedades y diferenciales.
- **Objetivos a Mediano Plazo (2027–2028) – Practicidad y Escalabilidad:**
    - Incorporación de topologías físicas y calibración de hardware en el prompt/modelo.

    - Reducción de la brecha simulación-hardware mediante reparación consciente del ruido y decodificación de corrección de errores asistida por LLM.
    - Detección estática y dinámica de vulnerabilidades en representaciones intermedias (QIR).
- **Objetivos a Largo Plazo (2029–2030) – Colaboración Humano-IA:**
    - Interfaces de desarrollo con "humano en el bucle" (*Human-in-the-loop*) para el descubrimiento exploratorio de nuevos algoritmos cuánticos.
    - Explicabilidad profunda del diseño de circuitos.
    - Métricas de cobertura cuántica escalables para verificar propiedades complejas como el entrelazamiento

# Beyond Rules: LLM-Powered Linting for Quantum Programs

**1. Motivación y Problema**

- **Inadecuación de las reglas fijas:** Los *linters* basados en reglas deterministas (como el estado del arte previo, **LintQ**) requieren mucho mantenimiento manual y tienen dificultades para adaptarse a las evoluciones rápidas de las APIs cuánticas y a problemas contextualmente complejos.
- **Solución propuesta:** Evolucionar la detección de problemas hacia razonamientos flexibles con LLMs mediante dos variantes avanzadas:
    1. **LintQ-LLM+CoT:** Implementa una secuencia estructurada de *Chain-of-Thought* (CoT) dividida en planificación estratégica, comprensión del código, lógica de detección e informe estructurado.
    2. **LintQ-LLM+RAG:** Agrega un sistema de Generación Aumentada por Recuperación (*Retrieval-Augmented Generation*) para conectar el razonamiento del LLM con una base de conocimiento vectorial (indexada con FAISS) formada por casos reales de prueba previamente verificados.

**2. Metodología y Diseño Experimental**

- **Categorías de problemas evaluados:** 10 tipos de patrones o problemas cuánticos (ej. aplicar puertas tras medir OpAfterMeas, registros clásicos insuficientes InsuffClasReg, o uso de APIs obsoletas OldIdenGate).
- **Corpus de evaluación:** 55 programas de Qiskit (43 archivos reales y 12 sintéticos con fallos inyectados para cubrir categorías raras).
- **Métricas:** Precisión (corrección), Recall (completitud) y F1-Score (efectividad global) comparados contra el *Ground Truth* validado manualmente y el *linter* basado en reglas **LintQ**.

**3. Resultados Clave**

- **Superación del estado del arte (RQ1):**
    - **LintQ (Reglas tradicionales):** F1-score de **0.41** (Precisión: 0.39, Recall: 0.43).

    - **LintQ-LLM+CoT:** F1-score de **0.70** (Precisión: 0.55, Recall: **0.96**).
    - **LintQ-LLM+RAG:** F1-score de **0.68** (Precisión: **0.56**, Recall: 0.86).
    - *Conclusión:* Los enfoques con LLM demuestran ser significativamente más efectivos y completos que las herramientas basadas en reglas.
- **Impacto del módulo RAG (RQ2):**
    - El uso de RAG genera un **intercambio entre precisión y sensibilidad**: reduce los falsos positivos aumentando ligeramente la precisión (0.56 vs 0.55), pero rigidiza la detección al basarse en plantillas conocidas, reduciendo el Recall (0.86 vs 0.96).
    - RAG es recomendable cuando se prioriza la precisión absoluta (evitar falsas alarmas), mientras que CoT puro es preferible para no dejar escapar ningún error (máxima completitud).
- **Análisis complementarios:**
    - **Resistencia a la ofuscación:** El modelo base razona principalmente sobre las propiedades topológicas del circuito cuántico y no sobre nombres de variables memorizados (descartando fuga de datos).
    - **Calidad vs. Cantidad en RAG:** La efectividad del RAG depende de la claridad sintáctica y estructural del tipo de error y no del volumen bruto de ejemplos en la base de datos.

**4. Conclusión**

Los LLMs (con técnicas CoT y RAG) proporcionan una base adaptable y escalable para la nueva generación de herramientas de aseguramiento de calidad de software cuántico, trasladando la carga de trabajo desde la redacción manual de reglas hacia la ingeniería de prompts y la curación de bases de conocimiento.

# Towards Quantum Program Bug Datasets and Benchmarking

**1. Motivación y Problema**

- **Ausencia de benchmarks cuánticos reproducibles:** A medida que el ecosistema de software cuántico madura, garantizar su confiabilidad es crítico. Sin embargo, la comunidad carecía de un benchmark estandarizado y ejecutable para evaluar la reparación de código cuántico.
- **Limitaciones de las métricas clásicas:** Las suposiciones de evaluación del software clásico no se aplican directamente al ámbito cuántico debido a:
    1. Resultados estocásticos/probabilísticos (mediciones de probabilidad en lugar de salidas fijas).
    2. Criterios de equivalencia basados en tolerancia numérica (valores esperados, estabilidad de punto flotante).
    3. Fragilidad del entorno de ejecución (sensibilidad a versiones de compiladores, simuladores y bibliotecas).
    4. Defectos multicapa/cruzados (interacciones entre compiladores, simuladores y código de aplicación).

**2. La Propuesta: QBench**

QBench recopila **520 tareas de reparación reales** procedentes de **129 repositorios** de software cuántico. Cada instancia empaqueta el entorno empaquetado/aislado, un oráculo de corrección y el protocolo de evaluación.

QBench se organiza en dos pistas complementarias:

- **RepoTrack (264 instancias):** Enfocada en errores de infraestructura (compiladores, simuladores, bibliotecas base). Suele requerir modificaciones en múltiples archivos y razonamiento a nivel de repositorio.
- **ScriptTrack (256 instancias):** Enfocada en programas cuánticos de usuario y flujos experimentales (algoritmos, scripts de prueba, entrenamiento variacional). Son parches más localizados pero con fuerte lógica de dominio.

**Soporte de Oráculos de Corrección:**

Para adaptarse a la semántica cuántica, QBench soporta tres tipos de oráculos:

1. **Deterministas:** Pruebas unitarias, excepciones y contratos de API.
2. **Con tolerancia numérica:** Verificación de equivalencia aproximada basada en umbrales.
3. **Probabilísticos:** Evaluación estadística mediante ejecuciones repetidas (*shots*) sobre distribuciones de medición cuántica.

**3. Principales Resultados de la Evaluación Empírica**

Los autores evaluaron baselines convencionales y un conjunto representativo de **Modelos de Lenguaje de Gran Tamaño (LLMs)** en configuraciones *one-shot* e *iterativas*:

- **Jerarquía de rendimiento (Pass@1 global):**
    - **LLMs Frontera Generales (GPT-4o, Claude-3.5-Sonnet, Gemini-2.5-Pro):** Alcanzaron las tasas más altas (hasta **50.1%** global: 38.6% en RepoTrack y 61.7% en ScriptTrack).
    - **LLMs Especializados en Código (DeepSeek-Coder, Qwen2.5-Coder, Codestral-22B):** Lograron entre 34.4% y 36.5% global.
    - **LLMs de Pesos Abiertos (LLaMA-3.1, DeepSeek-R1):** Lograron entre 29.1% y 39.7%.
    - **Baselines Tradicionales (Plantillas heurísticas y APR basado en búsqueda):** Rendimiento muy limitado (10.9% - 17.1%).
- **Dificultad por Tipo de Oráculo:**
    - Todos los modelos se desempeñaron mejor ante **oráculos deterministas** (más del 60% de éxito en modelos frontera).
    - El rendimiento cayó sustancialmente ante **oráculos probabilísticos** (<25% en RepoTrack para modelos frontera), ya que los modelos tienden a generar parches frágiles que satisfacen una ejecución individual pero fallan al evaluar distribuciones estadísticas repetidas.
- **Impacto de la Retroalimentación del Oráculo (Iteratividad):**
    - Permitir que el agente de LLM reciba mensajes de error y vuelva a intentar (hasta $B = 10$ iteraciones) casi duplicó el éxito en RepoTrack, ayudando a corregir errores de entorno, compilación y dependencias.

**4. Conclusiones**

QBench demuestra que la **naturaleza estocástica de los oráculos**, la **fragilidad del entorno de ejecución** y los **defectos entre capas de software** son los mayores obstáculos para la reparación automática de código cuántico. El benchmark se ha liberado públicamente para fomentar el desarrollo de técnicas de reparación adaptadas a la física y semántica de la computación cuántica.

# Generic or Quantum-Specific? Why the Control Matters: Difficulty-Matched Probes of LLM Quantum-Code Failures

**1. El Problema y la Aportación Metodológica**

- **La trampa del control inadecuado:** La mayoría de las evaluaciones sobre asumen que los fallos son cuánticos por naturaleza, pero no lo prueban. Para medir esto, se suele usar un "control estructural" clásico (un problema clásico con la misma forma o tamaño de matriz de salida).
- **Hallazgo central:** El tipo de control elegido determina el resultado. Si se usa un *control estructural simple*, casi todos los fallos del modelo parecen ser cuánticos. Sin embargo, ese control clásico es tan fácil que los modelos lo aprueban el 99.5% de las veces, sobreestimando falsamente la especificidad cuántica.
- **Solución (Control Ajustado por Dificultad):** El autor diseña una prueba apareada usando el **Transformada Rápida de Fourier clásica (FFT)** como grupo de control para la **Transformada de Fourier Cuántica de 3 qubits (QFT-3)**. Ambos comparten exactamente la misma dificultad algorítmica de composición ordenada de operaciones no conmutativas, pero la FFT no requiere física o semántica cuántica.

**2. Diseño Experimental**

- **Evaluación:** Se probaron **9 modelos** de 4 proveedores (incluyendo Claude Opus 4.7/Sonnet 4.6/Haiku 4.5; GPT-5.5/GPT-5.4-Mini; Gemini 2.5 Flash; Qwen2.5-Coder-32B y Llama-3.3-70B) a lo largo de **12 prompts apareados** en 5 niveles de dificultad.
- **Criterios de Evaluación:** Evaluados mediante simulación directa en Qiskit y clasificados mediante una taxonomía de tipos de fallo.

**3. Resultados Principales**

1. **Reevaluación de la Especificidad Cuántica (Caso QFT-3):**
    - Bajo el *control estructural simple*, el 100% de los fallos aparentaban ser déficits cuánticos específicos.

- Bajo el *control ajustado por dificultad (FFT)*, la especificidad cuántica real cae al **50%** (8 de 16 fallos).

2. **División según la Capacidad del Modelo:**
   - **Modelos avanzados (ej. GPT-5.4-Mini, Claude Haiku 4.5):** Aprueban la prueba clásica de la FFT pero fallan en la QFT-3, demostrando un **déficit verdaderamente cuántico**.
   - **Modelos más débiles (ej. Gemini 2.5 Flash-Lite, LLaMA-3.3-70B):** Fallan tanto en la QFT cuántica como en la FFT clásica, probando que su fallo es una **incapacidad genérica** para ordenar operaciones complejas y no un problema cuántico.
3. **Brecha de Inmensurabilidad (Tareas Nativas Cuánticas):**
   - En 44 de los 63 fallos cognitivos (ej. Estado de Bell, oráculo Toffoli, etc.), el objetivo clásico es trivial de escribir (un simple numpy.array), mientras que toda la dificultad reside en el conjunto de puertas cuánticas. Debido a que no existe un equivalente clásico complejo con el cual comparar, el autor concluye que en esos casos **no es posible medir la especificidad** y declara formalmente no atribuirlos como fallos cuánticos directos.
4. **Patrón Cognitivo Predominante:**
   - La causa raíz principal de los fallos es el uso de **"Plantillas Visuales sin Verificación Semántica"**. Los LLMs reproducen la apariencia sintáctica de un circuito o receta cuántica (ej. colocar puertas Hadamard seguidas de fases controladas), pero no verifican si la matriz unitaria resultante calcula realmente la transformación requerida.
5. **Efecto de la Retroalimentación (Agente de Corrección):**
   - Al dar retroalimentación de errores del simulador, las equivocaciones de *orden de puertas* (como en QFT-3) son altamente recuperables, mientras que los fallos de *compromiso con una receta errónea* (como en los oráculos de fase) persisten tras varios intentos de reparación.

**4. Conclusiones y Recomendaciones**

- **Atribución de fallos:** Afirmar que un LLM falla debido a la "física o lógica cuántica" exige comparar su rendimiento contra controles clásicos que posean la misma carga o dificultad algorítmica.

- **Límite de los modelos especializados:** El modelo Qwen2.5-Coder-32B (especializado en código) continuó teniendo fallos importantes frente a modelos generales de frontera (como Claude Sonnet 4.6 o GPT-5.5, que lograron un 100% de éxito), demostrando que la especialización sintáctica en código clásico no sustituye las capacidades de razonamiento abstracto de mayor escala.

# Are Large Language Models Ready for Quantum Software Engineering?

**1. Cobertura de las Tareas del Ciclo de Vida de QSE ($RQ_1$)**

Basándose en la taxonomía del ciclo de vida del software cuántico, los autores observan que la adopción de los LLMs está **fuertemente centrada en el código**, descuidando casi por completo las fases iniciales y finales:

- **Implementación (Fase dominante):** Centrada en la generación automática de circuitos cuánticos, código en Qiskit/PennyLane y la asistencia interactiva.
- **Análisis y Mantenimiento:** Apoyo en tareas de reparación de código (*bug fixing*), refactorización/migración entre versiones de SDKs, comprensión/explicación de algoritmos y herramientas de *linting* (análisis estático).
- **Pruebas (Testing) y Diseño:** Muy poco exploradas (solo un par de estudios sobre *fuzzing* o detección de pruebas *flaky* y diseño de *ansatz* arquitectónicos).
- **Fases Ausentes:** No se encontró evidencia del uso de LLMs en ingeniería de requisitos, despliegue o mantenimiento avanzado a nivel de infraestructura.

**2. Modelos Más Utilizados ($RQ_2$)**

- **Dominio de modelos propietarios:** La familia **GPT de OpenAI** (GPT-3.5, GPT-4, GPT-4o, GPT-4.1) domina abrumadoramente la literatura. Esto se debe a que la programación cuántica es una tarea compleja con escasez de datos, por lo que los investigadores prefieren utilizar los modelos comerciales más potentes disponibles.
- **Alternativas de código abierto:** Se observa un uso emergente pero fragmentado de modelos *open-weight* (como **LLaMA**, **DeepSeek**, **StarCoder**, **Granite**, **Qwen** y **Mistral**), usados a menudo de forma complementaria.
- **Riesgo metodológico:** La fuerte dependencia de modelos cerrados y cambiantes compromete la **reproducibilidad** y la comparación longitudinal de los experimentos a lo largo del tiempo.

**3. Limitaciones y Desafíos Detectados ($RQ_3$)**

La revisión agrupó los problemas reportados por la literatura en 8 grandes temas:

1. **Inexactitud Semántica y Alucinaciones:** Producción de circuitos sintácticamente válidos pero lógicamente incorrectos o inejecutables en procesadores reales.
2. **Escasez y Cobertura de Datos:** Los LLMs carecen de suficientes datos de entrenamiento sobre APIs cuánticas recientes, conceptos físicos y patrones avanzados.
3. **Degradación con la Complejidad:** Funcionan aceptablemente en tareas simples o de libro de texto, pero su rendimiento cae drásticamente ante problemas complejos de escala real.
4. **Alta Sensibilidad al Prompting:** El éxito depende enormemente de cómo se redacta el prompt, requiriendo ingeniería de prompts compleja.
5. **Problemas de Alineación y Explicaciones Superficiales:** Explicaciones de código que carecen de profundidad o no se alinean con la intención real del desarrollador.
6. **Limitaciones de Evaluación:** La mayoría de los estudios evalúan modelos en benchmarks simplificados o restrictivos.

**4. Conclusión del Estudio**

Los autores concluyen que **los LLMs NO están listos para actuar como agentes autónomos en todo el ciclo de vida de la Ingeniería de Software Cuántico**.

Actualmente funcionan solo como **herramientas de asistencia limitada** en tareas acotadas a nivel de código. Para que la industria los adopte de forma segura, el software generado por LLMs debe tratarse como un artefacto provisional que pase obligatoriamente por **puertas de validación estricta** (compilación, pruebas de simulación y ejecución en backends) antes de integrarse.

# How AI, LLMs and quantum science can empower each other?

### 1. Introducción y Fundamentos de los LLM

#### Principios de la IA Generativa y Chatbots LLM

- La IA generativa utiliza redes neuronales profundas (como la arquitectura *Transformer*, inventada en 2017) entrenadas en masivas cantidades de datos para extrapolar y generar nuevo contenido (texto, código, imágenes, etc.).
- Los LLM convierten texto en "tokens" y los representan como vectores de coma flotante en espacios multidemensionales (*embeddings*) para predecir probabilísticamente la continuación del texto.
- Los modelos no poseen un razonamiento deductivo o inductivo dinámico genérico; su "razonamiento" percibido se extrae de patrones estáticos presentes en sus datos de entrenamiento.

#### Técnicas de Optimización y Extensiones del Sistema

- **Chain of Thought (CoT) / Tree of Thoughts (ToT):** Controlan secuencias de pasos intermedios para guiar al modelo a descomponer problemas complejos, mejorando el razonamiento aparente.
- **Fine-Tuning e Instruction Tuning:** Ajustan los pesos neuronales (mediante métodos como LoRA) o la capacidad de seguir instrucciones específicas para especializarse en dominios concretos.
- **Reinforcement Learning from Human Feedback (RLHF) / DPO:** Alinean las respuestas del modelo con preferencias y reglas éticas humanas.
- **Retrieval-Augmented Generation (RAG):** Conecta el LLM a bases de datos vectoriales externas (ej., artículos de arXiv o documentos internos). Esto permite responder con información actualizada, reducir alucinaciones y citar fuentes explicables sin necesidad de reentrenar continuamente el modelo.

#### Propuesta de un LLM Especializado en Ciencia Cuántica

El autor propone formalmente la creación de un **chatbot basado en un LLM de código abierto especializado en ciencia cuántica**, alimentado vía RAG mediante la ingesta y vectorización del corpus científico del sector (por ejemplo, el repositorio completo de artículos cuánticos de arXiv).

## 2. Aplicaciones de los LLM en la Ciencia Cuántica

El estudio analiza el impacto de los LLM (principalmente GPT-4 y Google Bard/Gemini a fecha de finales de 2023 / principios de 2024) en diversas áreas del trabajo científico y educativo:

### A. Aprendizaje y Estudio de Conceptos Cuánticos

- **Desempeño:** Los LLM responden con precisión y evitan errores comunes al explicar conceptos teóricos generales (superposición, entrelazamiento, desigualdades de Bell).
- **Limitaciones:** Muestran dificultades en razonamientos específicos o al distinguir sutiles reglas físicas (por ejemplo, en pruebas de si el Rubidio-87 es un bosón o un fermión, donde Bard "alucinó" la respuesta tras la presión del usuario).
- **Exámenes:** Modelos como GPT-4 alcanzan calificaciones destacables en exámenes universitarios de física/cómputo cuántico (por ejemplo, un 69/100 en la prueba final de Scott Aaronson), pero fallan en preguntas abiertas o cálculos analíticos estrictos.

### B. Revisión de Artículos Científicos

- **Capacidades:** Ayudan a resumir rápidamente artículos científicos (arXiv), extraer puntos clave, verificar estructuras formales y evaluar metodologías.
- **Herramientas de apoyo:** Los visores de IA permiten a los revisores humanos ganar productividad comprobando la coherencia general, la bibliografía y detectando sesgos.

### C. Redacción y Gestión Científica

- Apoyan en la elaboración de solicitudes de financiación (*grants/RFPs*), resúmenes, textos para la creación de Políticas Nacionales de Cuántica (NQI) y cartas de presentación.
- Se advierte contra el uso indiscriminado en estudiantes, ya que puede mermar el pensamiento crítico, incentivar el "meta-plagio" y promover la propagación de ideas infundadas (*hype*).

### D. Desarrollo de Software

- **Productividad:** Altamente efectivos para escribir y optimizar código en Python (Qiskit, Cirq, etc.), crear expresiones regulares, estructurar scripts de procesamiento de datos o construir gráficos.
- **Asistentes de Código:** Aparición de herramientas integradas como *Qiskit Code Assistant* (IBM) y *Azure Quantum Copilot* (Microsoft).
- **Deficiencias:** Generan errores o inventan llamadas a APIs inexistentes (*alucinaciones de código*) cuando se les pide implementar lógicas algorítmicas complejas no disponibles masivamente en repositorios públicos.

**E. Ideación e Investigación Colaborativa**

- Actúan como "máquinas de hipótesis": no generan ciencia nueva por sí solas, pero hibridan conceptos de distintas disciplinas para sugerir nuevos enfoques en problemas específicos (como mitigar el ruido ambiental en qubits superconductores o descubrir nuevos materiales).
- Facilitan la comunicación interdisciplinar traduciendo la jerga técnica entre físicos experimentales, teóricos e ingenieros de software.

**3. Evaluaciones Empíricas y Muestras de Desempeño (Supplemental Materials)**

El autor incluye múltiples pruebas estructuradas con prompts reales donde destaca:

- **Excelente desempeño:** Estructurar hojas de datos técnicos/figuras de mérito (ej., fuentes y detectores de fotones individuales, amplificadores paramétricos TWPA, circuladores) y generar código de análisis de datos.
- **Fallas graves (Alucinaciones):**
    - **Citas falsas:** Google Bard inventó repetidamente referencias bibliográficas inexistentes al ser consultado sobre *bosonic qubits*.
    - **Generación de gráficos (DALL-E):** Intento fallido de generar diagramas de circuitos del algoritmo de Shor o esquemas de qubits; las imágenes generadas eran artísticas pero repletas de textos y conexiones absurdas sin sentido físico.
    - **Tablas comparativas de la industria:** Bard atribuyó fidelidades de puertas de 2 qubits a arquitecturas que no usan puertas (D-Wave) o a chips de control crio-CMOS (Intel Horse Ridge).

**4. Otras Tecnologías de Machine Learning en la Ciencia Cuántica**

Mientras que el papel de los LLM es asistencial/textual, otros algoritmos de *Machine Learning* clásico (redes neuronales convolucionales, recurrentes, de grafos, aprendizaje por refuerzo, redes de tensores) se utilizan activamente y con gran éxito en la investigación cuántica:

- **Física y Hardware:** Estimación del estado fundamental en sistemas de muchos cuerpos/espines, calibración dinámica en tiempo real de qubits, optimización de pulsos de microondas, trazado de rutas de trazado de qubits y mitigación de ruido.
- **Software y Herramientas:** Optimización de compiladores y transpiladores cuánticos mediante aprendizaje por refuerzo profundo; decodificación de síndromes de error en Códigos de Corrección de Errores Cuánticos (QEC) mediante *Transformers*; caracterización de detectores en sensores cuánticos y optimización de redes QKD.

**5. Aplicación del Cómputo Cuántico a la IA y los LLM**

En el análisis inverso —si la computación cuántica puede acelerar o mejorar la IA/LLM— el autor llega a conclusiones sobrias y escépticas a corto y medio plazo:

- **Quantum Machine Learning (QML):** En la era NISQ actual, QML (usando algoritmos variacionales como VQE/QAOA) no ha demostrado una ventaja cuántica real frente al cómputo clásico en datos de gran escala.
- **Inviabilidad en LLM:** Los LLM actuales (con cientos de miles de millones de parámetros y billones de tokens de entrenamiento) **no pueden procesarse en computadoras cuánticas** ni a corto ni a medio plazo.
    - *Razones:* Las computadoras cuánticas no están diseñadas para *Big Data*. La velocidad de entrada/salida de datos (velocidad de reloj cuántico y carga de estados) es sumamente lenta. Además, se requeriría una memoria RAM cuántica (qRAM) funcional, tecnología que aún no existe.
- **Algoritmos inspirados en la cuántica (*Quantum-Inspired*):** Algoritmos clásicos ejecutados en hardware convencional pero inspirados en matemáticas cuánticas (como la descomposición de redes de tensores) sí ofrecen beneficios reales hoy en día. Un ejemplo es la compresión de modelos de LLM (ej., *CompactifAI* de Multiverse Computing), que reduce el tamaño y la huella energética de los modelos clásicos.

**Resumen de Conclusiones Principales**

1. **Impacto Asimétrico:** La Inteligencia Artificial y el *Machine Learning* clásico están transformando profundamente la investigación, educación y desarrollo tecnológico de la ciencia cuántica. En cambio, que el cómputo cuántico beneficie a la IA o a los LLM es altamente incierto a corto y medio plazo.
2. **Rol del LLM:** Los LLM no reemplazan el juicio crítico ni el conocimiento del científico; actúan como potentes "asistentes/amplificadores". Es obligatorio el conocimiento previo del usuario para filtrar las "alucinaciones" y los sesgos mediante una "post-selección" de los resultados.
3. **Recomendación para el Ecosistema:** La comunidad científica cuántica debe aprender a dominar la ingeniería de prompts, adoptar flujos de trabajo basados en RAG y colaborar en la creación de herramientas de código abierto e infraestructuras de IA específicas para su dominio.